Kwame Nkrumah University of
Science & Technology, Kumasi, Ghana

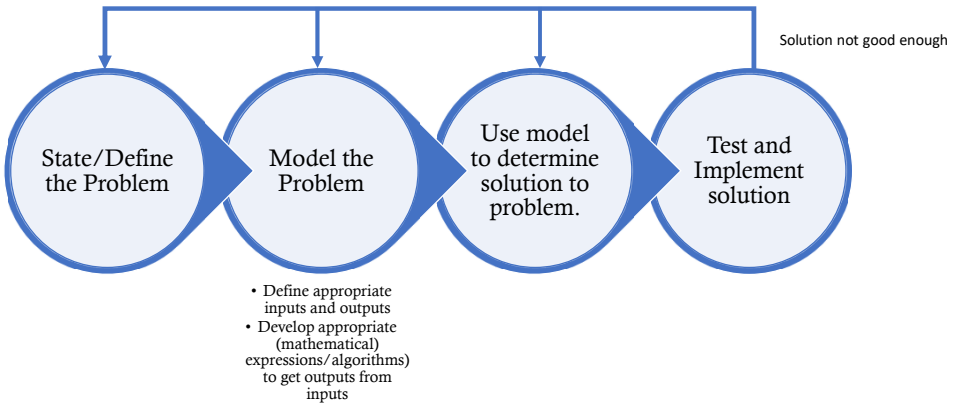
ME 168
COMPUTER PROGRAMMING FOR ENGINEERS (with MATLAB/OCTAVE)

D.E.K. Dzebre (PhD)

Department of Mechanical Engineering.

Introduction

- Problem solving in engineering (and several other fields) often follow a process similar to;



- Where mathematical model for a problem involves computations, computer computational tools greatly reduces computation times.

Introduction

- These tools are also often referred to as programming software or languages.
- There are a variety of programming software, often for different purposes.
- Softwares may be open-source or commercial.
- Commercially available software include MATLAB, Mathematica, MathCad, Maple, Stata, etc.
- Open-source tools include; python, R, Octave, SPSS, Stata, SAS, etc.

MATLAB

- The name MATLAB stands for MATrix LABoratory.
- The MATLAB® is high-performance language for technical computing, which integrates computation, visualization, and programming in an one environment.
- The basic data element for MATLAB is the array.
- Toolboxes (a family of add-on application-specific solutions) in MATLAB. allow you to learn and apply specialized technology.
- Toolboxes are comprehensive collections of MATLAB functions that extend the MATLAB environment to solve particular classes of problems.

GNU OCTAVE

- Conceived in 1988 as a course tool by the Universities of Wisconsin and Texas.
- Has evolved into a Free Scientific Programming Language, that runs on GNU/Linux, macOS, and Microsoft Windows.
- Octave syntax is largely compatible with MATLAB.
- Octave is extended with Octave Packages (somewhat similar to toolboxes in MATLAB)

MATLAB; Some uses

- Math and computation
- Modeling and simulation
- Data analysis, exploration, and visualization
- Scientific and engineering graphics
- Application development, including Graphical User Interface (GUI) building

Course Objectives

- At the end of the course you should be able to, (in MATLAB or GNU Octave);
  - assign numbers and strings to variables, vectors and matrices
  - create and retrieve data files
  - construct mathematical expressions
  - plot simple graphs
  - create and evoke M-files in the edit and command windows.
  - use built-in functions and write and use anonymous functions
  - create and access sub-functions
  - construct conditional statements using if…elseif and switch structures
  - construct loop statements using for…end and while statements

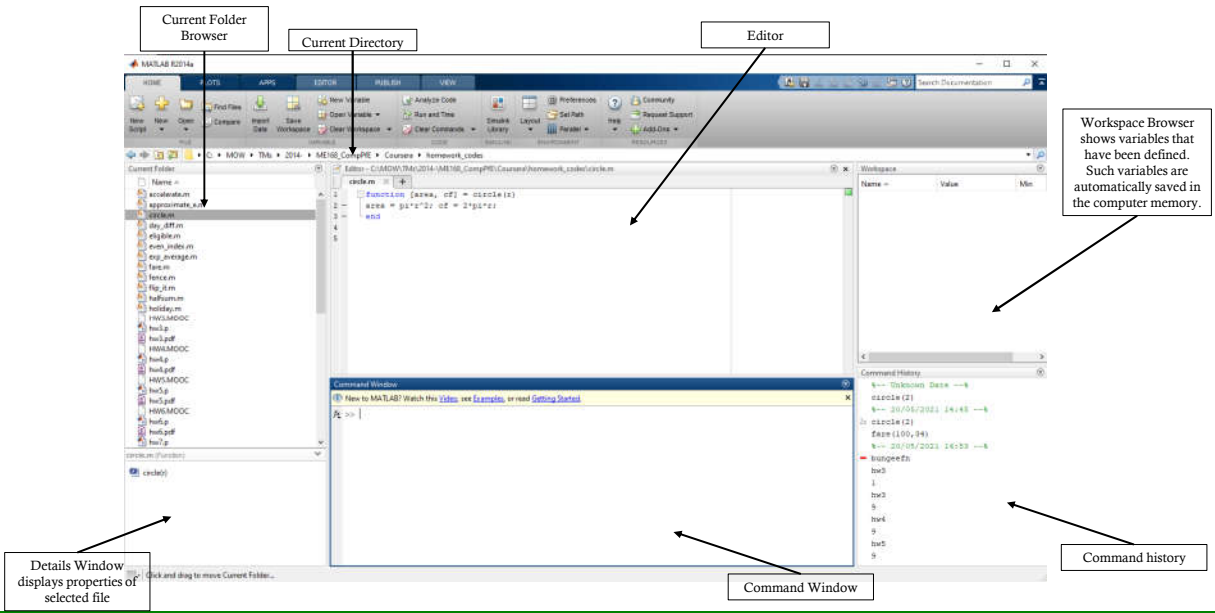
Outline

- Introduction to MATLAB (and GNU Octave);
- Basics: Matrices and Operators
- Plotting
- Scripts and Functions
- Selection
- Loops

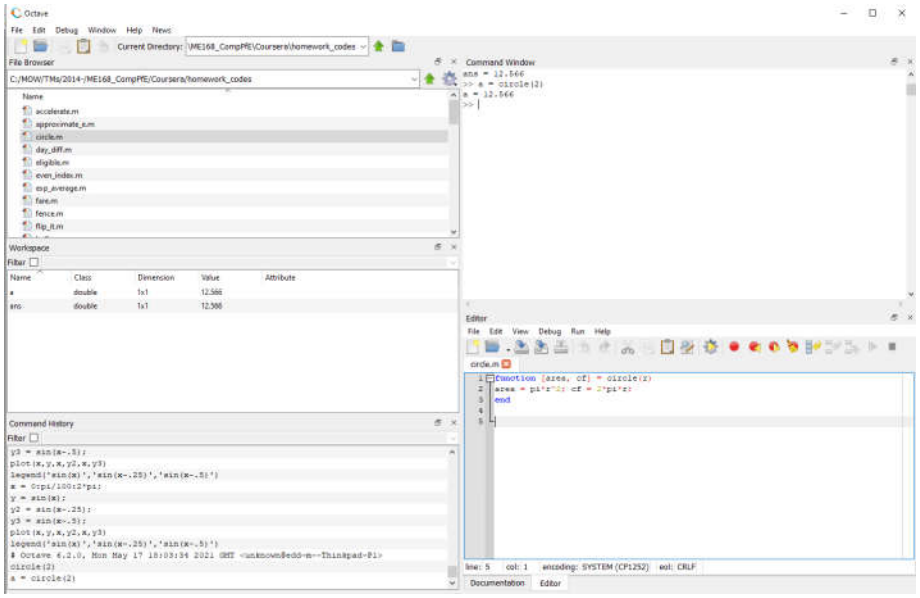
Course Assessment

- A

The MATLAB Interface



The Octave Interface



Expressions

- Scientific programming software can often be used as calculators.
- An expression is a formula consisting of **variables**, **numbers**, **operators**, and **function** names. E.g.

```
>> 2*pi
```

- Expressions are evaluated when entered into Matlab
- Expressions assigned to variables are called statements. E.g.

```
>> a = 2*pi
```

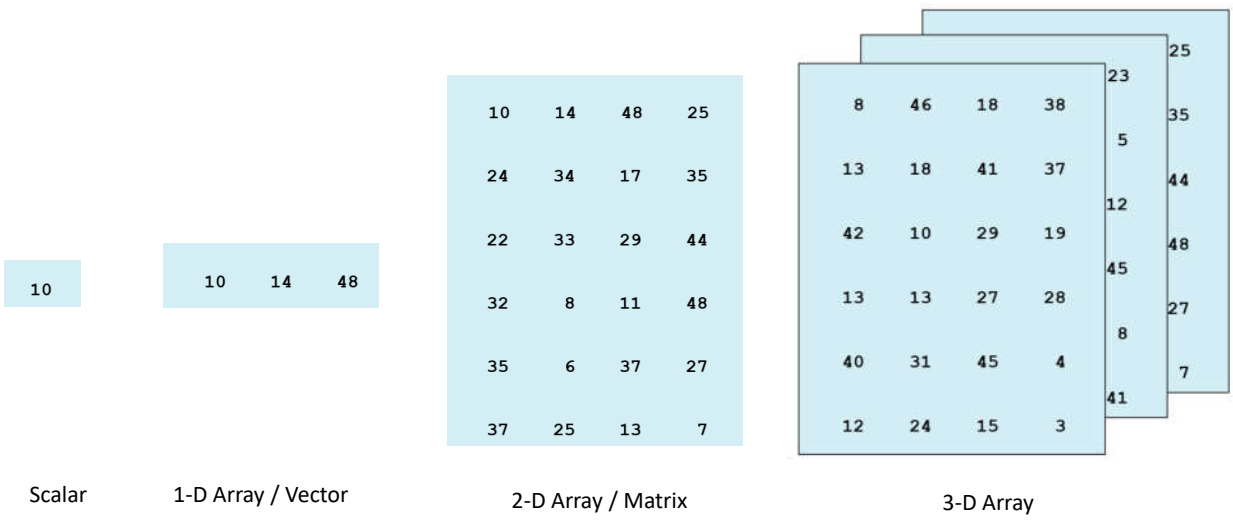
Variables and Arrays

- Variables are basically numbers to be used in computations that have been assigned names.

- In Matlab, variable names comply with certain rules
  - ✓It may consist only of the letters a–z, the digits 0–9 and the underscore (_).
  - ✓It must start with a letter.

- Variable names may be as long as you like, but MATLAB only remembers the first 63 characters.

Arrays



Scalar | 1-D Array / Vector | 2-D Array / Matrix | 3-D Array

Generating Arrays

Arrays can be generated in MATLAB by;

- Enter an explicit list of elements.
- Generate matrices using built-in functions.
- Load matrices from external data files.

Saving Variables Command Window Contents

Diary function

- Records a MATLAB session in a file (saved in the current directory)which can be retrieved for later review.
- Activated by command;

```
>> diary
      or
>> diary on
```

- To save to a custom filename;

```
>> diary('filename')    .E.g. diary('ME168')
```

- To turn off recording session

```
>> diary off
```

- Contents of file can be displaced in editor window for later review.

Saving Variables

- It is possible to save variables that have been created and recall them later into the workspace.
- This can be done with the command;

  >> `save` filename
- To save specific variables, use;

  >> `save` filename variable1 variable2 variable3
- To load variables into workspace, use load command

  >> `load` filename

Generating Arrays; Enter explicit list of elements

- Begin the matrix with left square bracket ([)
- Separate the elements of a row with blanks or commas.
- Use a semicolon(;) to indicate the end of each row, or press enter to enter the next row on a new line.
- End the entry with right square bracket (])
- Square brackets are not necessary if you are creating a scalar or a row vector

Generating Arrays; Enter explicit list of elements

```
>> A = [16 3 2 13; 5 10 11 8; 9 6 7 12; 4 15 14 1]
```

```
A =

    16     3     2    13
     5    10    11     8
     9     6     7    12
     4    15    14     1
```

4 x 4 matrix

- Use the Colon Operator (A=start:step:last)

```
>>  X = [1:4; 5:8; 9:12]
```

```
X =

     1     2     3     4
     5     6     7     8
     9    10    11    12
```

```
>> X = [1:.5:4; 5:.5:8; 9:.5:12]
```

```
X =

    1.0000    1.5000    2.0000    2.5000    3.0000    3.5000    4.0000
    5.0000    5.5000    6.0000    6.5000    7.0000    7.5000    8.0000
    9.0000    9.5000   10.0000   10.5000   11.0000   11.5000   12.0000
```


Generating Arrays; Enter explicit list of elements

- To determine the dimensions of a matrix (number of rows and columns)

```
>>  size(A)
```

- To determine the number of items in the longest dimension of a 2-D array

```
>>  length(A)
```

Generating Arrays; Built in functions

- All zeros: A= zeros (row, column). E.g.

```
>> A = zeros(2,3)
```

- All ones: A = ones (row, column) E.g.

```
>> A = ones(2,3)
```

- Identity matrices are generated with the "eye' function. E.g.

```
>> A = eye(2,3)
```

- The "rand" function generates uniformly distributed random elements (from 0 to 1). E.g.

```
>> A =rand (2,3)
```

- The "randn" command generates normally distributed random elements (from -1 to 1). E.g.

```
>> A = randn (2,3)
```


Generating Arrays; Read data from .txt and excel files

- In Matlab Text files should be organized as a rectangular table of numbers, separated by blanks, with one row per line, and an equal number of elements in each row.

- Data can be read from files with "load function".

```
>>  load('C:\MOW\TMs\2014-\ME168_CompPfE\ME168\data1.txt')
```

- From excel files, the "xlsread('filename')"

```
>>  xlsread('C:\MOW\TMs\2014-\ME168_CompPfE\ME168\data1.xlsx')
```

- Note: The package "io" must be loaded before the xlsread function can work in Octave. To load the package run;

```
>> pkg load io
```

Indexing

- Accessing parts of an array or sub-setting an array.

```
>>  X = [1:4; 5:8; 9:12]
```

- X(n,c) subsets the element on row n, and column c.

```
>> X(2,3)

ans =

     7
```

- Subset entire rows and columns

```
>> X(2,[1,2,3,4])

ans =

     5     6     7     8
```


Indexing (The "end" function)

- The end function returns the highest value taken on by that subscript. It can be used in sub setting entire rows and columns. For instance, if;

```
>> arr3 = [1 2 3 4 5 6 7 8]
```

`>> arr3(end)` would return the value 8

`>> arr3(5:end)` would be the array [5 6 7 8]

- And if

```
>>  X = [1:4; 5:8; 9:12]
```

`>> X(2,[1:end])` or `>> X(2,:)`

```
ans =

     5     6     7     8
```

`>> X([1,2,3],3)` or `>> X(:,3)`

```
ans =

     3
     7
    11
```

`>> X(end,end)` would return 12

Indexing

➢ Subset parts of an array to create another array.

If

```
>>  X = [1:4; 5:8; 9:12]

>> [X([1,2,3],1), X([1,2,3],4)]      or
>> X([1,2,3], [1,4])     or
 >> [X(:,1), X(:,end)]          will return
```

```
ans =

     1     4
     5     8
     9    12
```


Indexing

➢ Replace parts of an array

If

```
>>  X = [1:4; 5:8; 9:12]

>> X([1,2,3],1) = [13;14;15]
```

```
X =

    13     2     3     4
    14     6     7     8
    15    10    11    12
```

```
>> X(1,[1:end]) = [16 17 18 19]
```

Screen Display Format

- The format function controls the how numbers are displayed.
- Syntax: format ***type***
  - ***E.g*** ***format short***

Type	Result	Example
short	Scaled fixed-point format with 5 digits	3.1416
long	Scaled fixed-point format with 15 digits for double and 7 digits for single	3.14159265358979
short e	Floating format with 5 digits	3.1416e+000
long e	Floating format with 15 digits for double and 7 digits for single	3.14159265358979e+000
short g	Best of fixed- or floating-point with 5 digits	3.1416
long g	Best of fixed- or floating-point with 15 digits for double and 7 digits for single	3.14159265358979
short eng	Engineering format with at least 5 digits and a power that is a multiple of 3	3.1416e+000
long eng	Engineering format with at least 16 digits and a power that is a multiple of 3	3.14159265358979e+000
bank	Fixed dollar and cents or any bank equivalent	3.14
rat	Fractional form	355/113

Operators

Expressions use familiar arithmetic operators and precedence rules.

- = Assignment Operator
- + Addition
- - Subtraction
- * Multiplication
- / Division
- \ Left division
- .^ Power
- ‘ Transpose
- ( ) Specify evaluation order
- : Colon Operator

Other Array operations

- For individual matrices you can find
  - Transpose (‘)
  - Sum (sums each column)
  - Diagonal (diag)
  - Inverse (inv) etc

Operators (Addition)

- Operations between arrays, may be array operations or matrix operations.
- Array operations are carried out “element by element” and Matrix operations follow the normal rules of linear algebra. (e.g. matrix multiplication).
- Addition and subtraction and Exponentiation are array operators.
- Multiplication and division can be used as both array and matrix operators.

Operators (Addition)

- Z = X + Y means
  Z(m,n) = X(m,n) + Y(m,n) for all valid m and n

```
>> X = [1 5 -2; 3 0 7];
>> Y = [6 0 6; 2 2 1];
>> Z = X+Y = [6 0 6; 2 2 1]
```

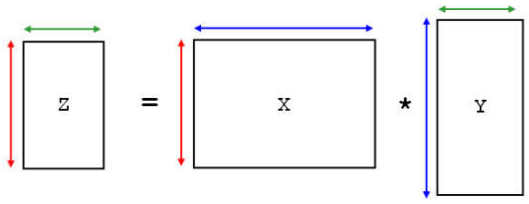
```
Z =

     7     5     4
     5     2     8
```

- Z = X - Y means
  Z(m,n) = X(m,n) - Y(m,n) for all valid m and n

Operators (Multiplication)

- Z =X*Y multiplication matrix operation



NOTE:
Inner dimensions of matrices must agree for this to work

- Z =X.*Y multiplicative array operation

Z(m,n) = X(m,n) x Y(m,n) for all valid m and n

```
>> X = [1 5 -2; 3 0 7]
>> Y = [6 0 6; 2 2 1]
>> Y = X.*Y
```

```
Z =

     6     0   -12
     6     0     7
```

Operators (Division and Exponentiation)

- Z =X/Y = Z*Y^{-1} (Matlab actually computes this as X/Y = (Y'\X')'

- Z =X./Y multiplicative scalar (or element-by-element operation)
  Z(m,n) = X(m,n) ÷ Y(m,n) for all valid m and n

- Z =X.\Y multiplicative scalar (or element-by-element operation)
  Z(m,n) = Y(m,n) ÷ X(m,n) for all valid m and n

- Z =X.^Y
  Z(m,n) = X(m,n) ^ Y(m,n) for all valid m and n

Precedence (Hierarchy of Operations)

- Operations are carried out not necessarily in the order in which they occur in the expression.
- Operators are ranked so that some act before others, regardless of the order in which they appear.

PRECEDENCE	OPERATOR
0	Parentheses: (...)
1	Exponentiation ^ and Transpose '
2	Unary +, Unary –, and logical negation: ~
3	Multiplication and Division (array and matrix)
4	Addition and Subtraction
5	Colon operator :

Precedence; Entering Expressions

- Often, a single expression will comprise several arithmetic operations on several variables.

- Evaluation will be carried out according to the precedence order of operators. So it is important to know how you want the expression to be evaluated.

- For instance,

*distance = 0.5 * accel * time^2* will be evaluated differently in each of the following cases;

```
>>  distance = 0.5 * accel * (time^2)

>> distance = (0.5 * accel * time)^2
```